

Výstupy ŠVP

INF-08e Vytvořím přehledný program pro vyřešení konkrétního problému s ohledem na jeho možné důsledky a svou odpovědnost za ně. (*Informatika 4, 2/2 · z INF-08*)

INF-06 Analyzuji problém, rozdělím problém na menší části, rozhodnu, které je vhodné řešit algoritmicky, své rozhodnutí zdůvodním; sestavím a zapíšu algoritmy pro řešení problému. (*Informatika 4, 2/2*)

INF-09 Ověřím správnost, najdu a opravím případnou chybu v algoritmu, otestuji, odladím a optimalizuji program. (*Informatika 4, 2/2*)

Učivo: vývoj programu – volba nástroje, návrh uživatelského rozhraní, nápověda a dokumentace, autorství a licence, etika programátora

Připomenutí

Začni problémem a uživatelem, ne technologií: kdo, s čím a proč?

Nejdřív nejmenší funkční verze, pak vylepšuj po malých krocích a po každém kroku spusť testy.

Rozděl program na funkce; hlavní část jen volá funkce a čte se jako obsah knihy.

Závěrečný programátorský projekt: vlastní přehledný program pro konkrétní problém, který navazuje na programy z týdnů 21–30. Úlohy A a B tě provedou čitelností, rozhraním, testy, dokumentací a licencí, úlohy C reflexí dopadů a prezentací. Hotový projekt ověř podle kontrolního seznamu v Must know. Podrobnosti najdeš v listu Must know.

A. Rozcvička

1. Co program vypíše? Přepiš ho čitelně: výstižné názvy funkce, parametrů a proměnných a docstring. Výstup se nesmí změnit. ★☆☆

```
def f(a, b):
    c = 0
    for i in a:
        if i >= b:
            c += 1
    return c

print(f([45, 38, 50, 29], 40))
```

2. Které komentáře jsou užitečné a které zbytečné? a) `x = x + 1` # přičti jedna; b) `# ceny ukládám v haléřích (int)`, aby nevznikaly chyby zaokrouhlení float; c) `# tady je cyklus`; d) `# D == 0` porovnávám s tolerancí, vstupy jsou desetinná čísla; e) `# TODO: ošetřit prázdný seznam`. ★☆☆
3. Vyber licenci: a) chci, aby můj program mohl kdokoli použít i upravit, jen musí uvést mé jméno; b) chci, aby i vylepšené verze zůstaly volně dostupné se zdrojovým kódem; c) k projektu patří návod a obrázky, které smí kdokoli sdílet s uvedením autora; d) nechci nikomu nic dovolit. ★☆☆
4. Co musíš udělat, když do projektu: a) zkopíruješ funkci z blogu, u které není uvedena licence; b) použiješ kód, který ti vygenerovala umělá inteligence; c) ti spolužák napsal jednu funkci; d) použiješ knihovnu s licencí MIT? ★★☆☆
5. Porovnej dvě verze programu. Uveď aspoň čtyři vylepšení verze B z pohledu uživatele. ★☆☆

```
# verze A
x = float(input())
print(x * 1.21)

# verze B
cena = float(input("Zadej cenu bez DPH v Kč: ").replace(",", "."))
print(f"Cena s DPH 21 %: {cena * 1.21:.2f} Kč")
```

6. Funkce pro rozhraní programu: Co program vypíše, když uživatel postupně zadá abc, -5 a 12,5? Proč je výhodné mít kontrolu vstupu v jedné funkci? ★★☆☆

```
def nacti_kladne_cislo(vyzva):
    while True:
        try:
            x = float(input(vyzva).replace(",", "."))
        except ValueError:
            print("To není číslo, zkus to znovu.")
            continue
        if x > 0:
            return x
        print("Číslo musí být kladné.")

print(nacti_kladne_cislo("Délka trasy v km: "))
```

7. Vyber vhodný nástroj a zdůvodni: a) evidence výsledků třídního turnaje s grafem; b) hra pro děti z 1. stupně; c) simulace s tisíci opakováními; d) měření teploty ve třídě každých 10 minut; e) přihlášky na školní akci. ★☆☆
8. Celé README projektu zní: „Kalkulačka cesty. Spusť python cesta.py.“ Co v něm chybí? Napiš osnovu kvalitního README. ★☆☆

B. Slovní úlohy

9. Výběr projektu: Napiš tři kandidátní problémy z domova, školy nebo koníčku (můžeš rozšířit program z týdnů 21–30: řešič rovnic, statistika z dat, simulace, generátor hesel...). U každého posuď, zda a proč je vhodný k algoritmickému řešení a zda ho stihneš. Jeden vyber. ★☆☆
10. Specifikace: Pro svůj projekt zapiš vstupy (typ, rozsah, jednotky), výstupy, podmínky řešení a co program neumí. Vzorem je specifikace v Must know. ★★☆☆
11. Rozklad a algoritmus (INF-06): Rozděľ svůj program na funkce (název, parametry, návratová hodnota) a hlavní část. Pro nejsložitější funkci napiš pseudokód nebo nakresli vývojový diagram. ★★☆☆
12. Část projektu: Funkce vazeny_prumer vrací vážený průměr známek. Co program vypíše? Proč funkce vrací None a kdy? Doplň další dva testy. ★★☆☆

```
def vazeny_prumer(znamky, vahy):
    """Vážený průměr; None, pokud je součet vah nulový."""
    if len(znamky) != len(vahy) or sum(vahy) == 0:
        return None
    soucet = 0
    for i in range(len(znamky)):
        soucet += znamky[i] * vahy[i]
    return soucet / sum(vahy)

assert vazeny_prumer([1, 2, 1, 3], [3, 1, 2, 1]) == 10 / 7
assert vazeny_prumer([], []) is None
print(round(vazeny_prumer([1, 2, 1, 3], [3, 1, 2, 1]), 2))
```

13. Testovací sada: Pro výpočetní část vzorového projektu (funkce cena_cesty) urči očekávané výstupy pro (400 km, 6,5 l/100 km, 38 Kč/l, 4 osoby) a (1 250 km, 7,2 l/100 km, 39,9 Kč/l, 3 osoby), pak program spusť. Doplň aspoň čtyři další testy pro rozhraní (hraniční a nesmyslné vstupy). ★★☆☆

```
def spotreba_litru(km, l_na_100):
    return km * l_na_100 / 100

def cena_cesty(km, l_na_100, cena_l, osob):
    litry = spotreba_litru(km, l_na_100)
    celkem = litry * cena_l
    return litry, celkem, celkem / osob

assert cena_cesty(400, 6.5, 38, 4) == (26.0, 988.0, 247.0)
assert cena_cesty(100, 5, 40, 1) == (5.0, 200.0, 200.0)
print(cena_cesty(400, 6.5, 38, 4))
print(cena_cesty(1250, 7.2, 39.9, 3))
```

14. Refaktoring: Program třikrát opakuje stejných 8 řádků pro načtení kladného čísla (km, spotřeba, cena). Nahraď je funkcí `nacti_kladne_cislo` z úlohy 6. O kolik řádků bude program kratší? Jak ověříš, že se chování nezměnilo? ★★☆☆
15. Dokumentace: Napiš docstring pro funkci `cena_cesty` (co dělá, parametry s jednotkami, co vrací) a README svého projektu podle osnovy z úlohy 8. ★☆☆
16. Verze a bezpečnost projektu (H61, H62): Proč ukládat průběžné verze a jak (git, kopie s datem)? Jak projekt zálohovat? Co nesmí být v kódu, který zveřejníš nebo sdílíš se spolužáky? ★☆☆

C. Problémy a propojení

17. Reflexe dopadů: Posuď tři programy – a) aplikace, která z fotky obličeje odhadne věk; b) bot, který automaticky lajkuje a komentuje příspěvky; c) program, který rozdělí žáky do skupin podle známek. Komu mohou pomoci, komu uškodit a jaká opatření navrhneš? Stejnou reflexi (5–8 vět) napiš ke svému projektu. ★★★
18. Odpovědnost za chybu (H47): Program pro zahrádkáře počítá dávku hnojiva, ale uživatel zadá množství v gramech místo kilogramů a program doporučí tisícinásobnou dávku. Kdo za škodu může? Navrhni čtyři úpravy programu, které takové chybě zabrání. ★★☆☆
19. Přístupnost: Tvůj program bude používat i nevidomý uživatel se čtečkou obrazovky, barvoslepý spolužák a babička, která s počítačem pracuje málo. Navrhni pro každého aspoň jednu úpravu rozhraní. ★★☆☆
20. Prezentace a zpětná vazba: Připrav tříminutovou prezentaci projektu (problém – ukázka běhu – jedna věc, která se nepovedla a jak jsi ji vyřešil – co bys udělal příště). Při prezentaci spolužáka zapiš dvě silné stránky a jedno doporučení podle kontrolního seznamu v Must know. ★★☆☆

Hotové úlohy k procvičení na internetu

- **Nauč se Python (PyLadies) – Spolupráce a open source** – svobodný software, volba licence (MIT, GPL) a soubor LICENSE, co patří do README
<https://naucese.python.cz/course/pyladies/git/collaboration/>
- **Creative Commons Česká republika – Licence CC** – prvky BY, SA, NC, ND a šest licencí pro texty, obrázky a další díla
<https://www.creativecommons.cz/licence-cc/>
- **Nauč se Python (PyLadies) – Testování** – automatické testy `assert` a `pytest` pro testovací sadu projektu
<https://naucese.python.cz/course/pyladies/beginners/testing/>

★ základní · ★★ střední · ★★★ náročnější. Vyber si, které úlohy spočítáš; řešení ukládej do svého portfolia.

1. 2; např. def pocet_uspesnych(body, hranice): ""Vrátí počet testů, ve kterých žák získal aspoň hranice bodů.""
... pocet += 1 ... return pocet – vypíše opět 2
2. užitečné: b), d) (vysvětlují proč), e) (připomínka nedokončené práce); zbytečné: a), c) – opakují, co je vidět z kódu
3. a) MIT; b) GPL; c) Creative Commons CC BY (případně CC BY-SA); d) nemusím nic uvádět – bez licence platí „všechna práva vyhrazena“ (nikdo ho ale pak nesmí legálně šířit ani upravovat)
4. a) uvést zdroj; pro zveřejnění projektu nemám právo cizí kód šířit – napsat vlastní řešení nebo najít kód s licencí; b) označit, které části jsou z AI (nástroj, zadání), zkontrolovat je, otestovat a umět je vysvětlit; c) uvést jeho autorství (spoluautor, poděkování); d) uvést knihovnu a ponechat text její licence
5. výzva říká, co zadat a v jakých jednotkách; povolena desetinná čárka; výstup má popis, sazbu a jednotku; zaokrouhlení na haléře (pro 100 → 121.00 Kč místo 121.0); dalším krokem je ošetření nečíselného vstupu
6. To není číslo, zkus to znovu. / Číslo musí být kladné. / 12.5 (výzva se pokaždé zopakuje); jedna funkce se použije pro všechny vstupy (km, spotřeba, cena) – kód se neopakuje a oprava se udělá na jednom místě
7. a) tabulkový procesor (data, vzorce, grafy); b) Scratch (grafika, postavy, souběh); c) Python (rychlé cykly, náhoda, měření času); d) micro:bit nebo čidlo s Pythonem a export do tabulky; e) online formulář – a promyslet, jaké osobní údaje sbírám a kdo k nim má přístup
8. chybí: co program přesně dělá a pro koho, vstupy a výstupy s jednotkami, příklad použití, omezení (výsledek je odhad), požadavky (verze Pythonu), autor, licence, datum nebo verze; osnova: název – účel – spuštění – příklad – omezení – autor a licence
9. např. vážený průměr známek s předpovědí potřebné známky; kvíz na slovíčka s náhodným pořadím a skóre; statistika tréninků (průměr, maximum, graf); kritéria: jasná pravidla a data (algoritmizovatelné), skutečný uživatel, rozsah na 2–4 hodiny práce
10. tabulka vstupů a výstupů, např. známky 1–5 (int), váhy 1–10 (int), výstup průměr na 2 desetinná místa; podmínky: seznamy stejně dlouhé, součet vah > 0; omezení: neukládá data mezi spuštěními
11. seznam 3–6 funkcí s hlavičkami (např. nacti_znamky()) → seznam, vazeny_prumer(znamky, vahy) → float nebo None, potrebna_znamka(...) → int) a hlavní část, která je jen volá; pseudokód s cyklem a větvením
12. $1.43 \left(\frac{3 + 2 + 2 + 3}{7} = \frac{10}{7} \right)$; None pro různě dlouhé seznamy nebo nulový součet vah (dělení nulou); další testy např. vazeny_prumer([5], [2]) == 5 a vazeny_prumer([1, 2], [1]) is None
13. (26.0, 988.0, 247.0); (90.0, 3591.0, 1197.0) – tj. 90 l, 3 591 Kč, 1 197 Kč na osobu; rozhraní: 0 km, -5, „abc“, desetinná čárka „6,5“, 0 osob (dělení nulou!) → výzva k opravě
14. místo $3 \cdot 8 = 24$ řádků je funkce (asi 9 řádků) + 3 volání = 12 řádků → asi o 12 řádků kratší; chování ověřit stejná testovací sada před a po úpravě (INF-09)
15. např. ""Spočítá spotřebu paliva (l), cenu cesty (Kč) a cenu na osobu. km – délka trasy v km, l_na_100 – spotřeba v l/100 km, cena_l – cena v Kč/l, osob – počet osob (> 0).""; README se všemi částmi osnovy
16. verze umožní vrátit se k funkčnímu stavu po chybné úpravě a ukázat postup práce; záloha 3-2-1 (počítač, cloud nebo repozitář, flashka); v kódu nesmí být hesla, přístupové klíče ani osobní údaje (jména a známky spolužáků) – testovací data vymysli
17. a) pomůže např. při kontrole věku, uškodí chybami (u některých skupin lidí větší), citlivá biometrická data – opatření: výslovný souhlas, zpracování v zařízení, neukládat fotky; b) „pomůže“ jen autorovi, klame ostatní a zkresluje algoritmy sítě (H62) – eticky nepřijatelné, porušuje podmínky sítě; c) pomůže učitelům, ale může žáky nálepkovat a zveřejnit známky – opatření: jen pro učitele, bez zveřejnění, možnost ručně upravit
18. odpovědnost nese i autor – program měl chybu předvídat; úpravy: jednotky přímo ve výzvě („v kg“), kontrola rozumného rozsahu s upozorněním, souhrn zadání před výpočtem („zadal jsi 5000 kg – je to správně?“), test hraničních a nesmyslných hodnot, upozornění „orientační výpočet“ v dokumentaci
19. nevidomý: čistý textový výstup bez „obrázků z hvězdiček“, informace nesdělovat jen rozložením, ovládání klávesnic; barvoslepý: nesdělovat význam jen barvou (text „chyba“, symboly); babička: jednoduché výzvy s příkladem („např. 6,5“), povolená čárka, vlídné chybové hlášky, málo kroků
20. osnova 4 bodů s časy (30 s problém, 90 s ukázka, 30 s chyba a ladění, 30 s poučení); zpětná vazba konkrétní a podle kritérií, např. „rozhraní ošetřuje čárku i text; doporučuji doplnit do README omezení“

Úloha 17

1. U každého programu si polož tři otázky: komu pomůže, komu může uškodit (chybou i zneužitím), jaká data zpracovává.
2. a) Odhad věku: pomůže např. při kontrole věku; škodí chybné odhady (u některých skupin lidí častější) a jde o citlivá biometrická data → výslovný souhlas, zpracování přímo v zařízení, fotky neukládat.
3. b) Bot na lajky a komentáře: „pomůže“ jen autorovi, klame ostatní a zkresluje doporučovací algoritmy (H62), porušuje podmínky sítí → eticky nepřijatelné, program nepsat.
4. c) Rozdělení do skupin podle známek: pomůže učitelům, ale může žáky nálepkovat a prozradit jejich výsledky → jen pro učitele, nic nezveřejňovat, možnost ručně upravit.
5. Vlastní reflexe: stejné tři otázky pro svůj projekt a konkrétní opatření, která jsi udělal(a) (kontrola vstupu, žádná osobní data, upozornění na omezení v README).